

Cyber Mfukoni (Guardian) — Complete App Description

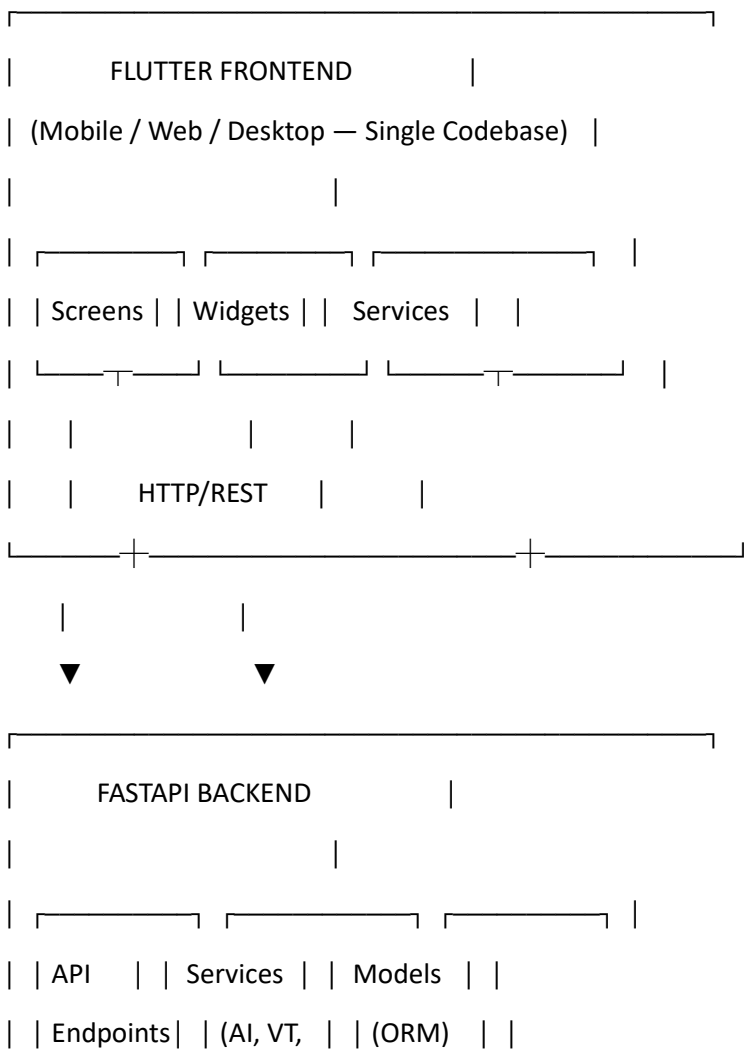
What Is It?

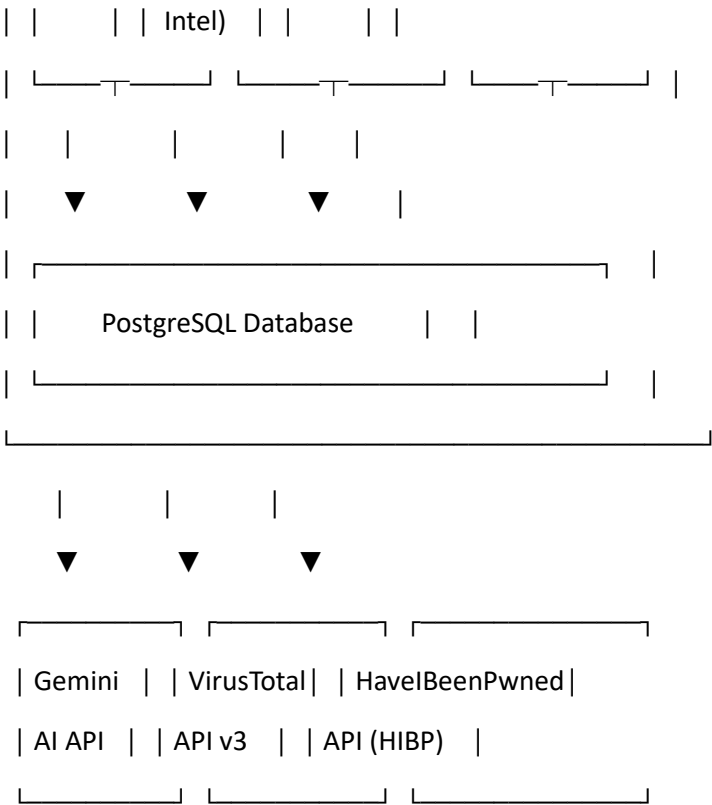
Cyber Mfukoni (Swahili for "*Cyber in Your Pocket*") is an AI-powered mobile cybersecurity platform designed to protect everyday users from digital threats. It combines real-time scam detection, threat intelligence, secure data storage, device scanning, and cybersecurity education into a single, premium-designed application. Think of it as a **personal cybersecurity bodyguard** that lives on your phone.

The app targets the growing need for accessible cybersecurity tools, especially in regions where mobile-first users are heavily targeted by phishing, SIM swap fraud, social engineering, and mobile money scams.

Architecture Overview

Cyber Mfukoni follows a **client-server architecture** with a clear separation between a Flutter mobile frontend and a Python REST API backend.





Tech Stack

Frontend

Technology	Purpose
Flutter (Dart)	Cross-platform UI framework (single codebase → Android, iOS, Web, Desktop)
Google Fonts	Typography (Inter, Share Tech Mono)
flutter_secure_storage	AES-256 encrypted key-value storage for vault data
local_auth	Biometric authentication (fingerprint / face ID)
image_picker	Camera and gallery access for photo imports
file_picker	Document/file selection for vault storage
path_provider	Cross-platform file system paths
crypto	SHA-1 hashing for HavelBeenPwned password checks
installed_apps	Fetches real installed Android packages for device scanning
cached_network_image	Efficient image loading/caching for Intel feed

Technology	Purpose
url_launcher	Opening external links (news articles, settings)
app_settings	Opens native device settings pages (Wi-Fi, Bluetooth, etc.)

Backend

Technology	Purpose
Python 3.11+	Backend language
FastAPI	Async REST API framework with auto-generated OpenAPI docs
SQLAlchemy	ORM for database models and queries
PostgreSQL	Primary relational database
Alembic	Database migrations
Pydantic	Request/response validation and settings management
Passlib + bcrypt	Password hashing for user authentication
python-jose	JWT token generation and verification
httpx	Async HTTP client for external API calls
APScheduler	Background job scheduling (Intel feed refresh)
DuckDuckGo Search	Cybersecurity news aggregation

External API Integrations

API	Module	Purpose
Google Gemini AI	Mulika AI + Cyber Agent	Scam message analysis, AI chat responses
VirusTotal v3	Device Scanner	Checks installed apps / file hashes against 70+ antivirus engines
HaveIBeenPwned	Dark Web Scanner	Password breach detection using k-Anonymity (SHA-1 range queries)
DuckDuckGo News	Intel Feed	Aggregates real-time cybersecurity news articles

Database Schema

Table	Key Fields	Purpose
users	id, email, username, hashed_password, total_score, streak	User accounts and gamification state
quiz_attempts	id, user_id, score, level, played_at	Chonjo game score history
scam_analyses	id, user_id, message_content, scam_probability, risk_rating, red_flags, explanation	Persisted Mulika scan results
chat_messages	id, user_id, role (user/agent), content, timestamp	Agent conversation history
intel_articles	id, title, summary, source_name, source_url, image_url, severity, published_at	Cached cybersecurity news
reported_numbers	id, phone_number, scam_category, details, report_count, last_reported_at	Community-reported scam phone numbers

Modules — How Each One Works

1. 🏠 Dashboard (Home)

What it does: Central hub that displays the user's security posture at a glance.

How it works:

- Shows a **Vault Health** circular progress indicator that calculates an overall security score based on Boma checklist completion.
- Displays quick-access cards to each major module (Mulika AI, Intel, Vault, Chonjo, Agent).
- Each card is tappable and navigates to the respective module screen.
- Uses glassmorphism design with gradient backgrounds, backdrop blur, and ambient glow effects.

2. 🔍 Mulika AI (Scam Detection Engine)

What it does: Analyzes suspicious messages, URLs, emails, images, and documents to determine if they're scams.

How it works:

- User selects an input type via scrollable chips: **SMS, Email, URL, QR Code, Image, Document, or Device Scan**.
- For text-based inputs (SMS, Email, URL): user pastes the suspicious content into a text field.
- For visual inputs (Image, QR Code, Document): user uploads a photo from camera or gallery. The image is base64-encoded.
- On "Analyze Threat", the app sends the content to POST /api/mulika/analyze.

- The backend passes the content to **Google Gemini AI** with a cybersecurity-focused system prompt.
 - Gemini returns a structured JSON response with: `scam_probability` (0–100), `risk_rating`, `red_flags[]`, `confidence`, `explanation`, and `community_reports`.
 - The frontend renders an animated threat assessment card with a circular score indicator color-coded by severity (green → red).
 - All analyses are persisted to the `scam_analyses` database table linked to the user.
-

3. 📱 Device Scanner (VirusTotal Integration)

What it does: Scans the phone for malware by checking installed apps against VirusTotal's database of 70+ antivirus engines.

How it works:

- Lives as a sub-mode within the Mulika screen (the "Device Scan" chip).
 - User selects a scan target: **Apps, Files, Storage Devices, Whole Phone, or Target Location**.
 - On **Android (native)**: the `installed_apps` package queries the Android PackageManager to get a real list of all installed apps (package names like `com.whatsapp`).
 - On **Web/Desktop**: generates realistic mock package names for prototype demonstration.
 - A cyberpunk-themed **hacker terminal** animates each package being scanned in real-time with green monospace text.
 - The package names are sent to the backend endpoint `POST /api/mulika/scan-apps`.
 - The backend's `virustotal_service.py` queries VirusTotal's Intelligence Search API (`GET /api/v3/intelligence/search`) for each package name.
 - VT returns detection stats (how many of 70+ engines flagged it as malicious).
 - The backend aggregates results into a threat report with a risk score.
 - If the VT API key is not configured, the app gracefully falls back to a local heuristic analysis.
 - Rate limiting is enforced (1-second delay between requests) to stay within VT's free tier (4 req/min).
-

4. 📰 Intel Feed (Threat Intelligence)

What it does: Real-time cybersecurity news and threat alerts from across the internet.

How it works:

- Backend uses **APScheduler** to periodically (every 3 hours) fetch cybersecurity news via **DuckDuckGo Search API**.

- Articles are deduplicated by title, classified by severity (Critical/High/Medium/Low), and cached in the intel_articles PostgreSQL table.
 - The Flutter frontend fetches articles from GET /api/intel/feed and displays them as cards with: title, source, severity badge, summary, publication date, and thumbnail.
 - Pull-to-refresh triggers a fresh fetch.
 - Articles are tappable and open the full source URL in the system browser.
-

5. Boma (Device Security Audit)

What it does: Performs a comprehensive security checklist audit of the user's device, scores it, and tells them exactly what to fix.

How it works:

- Contains a checklist of 10+ security items across categories: **Device, Accounts, Network, Privacy**.
 - Some items are **auto-detected** using native APIs:
 - Screen Lock: checked via local_auth (queries device biometric/PIN status).
 - Biometric Auth: checked via LocalAuthentication.getAvailableBiometrics().
 - Other items are **self-reported** by the user (e.g., "I use a VPN", "I have 2FA enabled").
 - Each completed item adds to an overall **security score** (0–100%) displayed as an animated circular gauge.
 - Tapping any checklist item opens the relevant **system settings page** (e.g., tapping "Screen Lock" opens the device's lock settings via app_settings).
 - Includes a link to the **Protection Guides** screen for step-by-step hardening tutorials.
-

6. Protection Guides

What it does: Step-by-step educational guides teaching users how to protect themselves from specific cyber threats.

How it works:

- Six illustrated guides: **SIM Swap Protection, Social Media Safety, Banking Security, Identity Theft Guard, Password Health, Wi-Fi Security**.
 - Each guide opens a detailed step-by-step walkthrough screen (GuideStepsScreen) with numbered instructions.
 - Uses custom WebP background images and color-coded accent themes per guide.
 - Fully responsive layout — card grid on mobile, wrap layout on desktop.
-

7. Secure Vault

What it does: A private, encrypted vault for storing sensitive data — functions like a combination of Google Password Manager, a private photo gallery, a secure notepad, and a hidden file safe.

How it works:

- **Storage Engine:** VaultStorageService uses flutter_secure_storage (backed by Android Keystore / iOS Keychain / Windows DPAPI) to store encrypted JSON metadata.
- **Cross-platform architecture:** Uses Dart conditional imports (vault_storage_native.dart / vault_storage_web.dart) to handle platform differences:
 - **Native (Android/iOS/Desktop):** Files are physically copied to a hidden directory (getApplicationDocumentsDirectory()/.cyber_vault/) with obfuscated UUID filenames.
 - **Web:** File bytes are base64-encoded and stored entirely within browser secure storage.

Sub-modules:

Module	What It Stores	How It Works
Passwords	Website, username, password	Add/view/delete. Password is masked by default with a show/hide toggle. Works like Google Password Manager.
Credit Cards	Card number, expiry, CVV, cardholder name	Masked display (shows last 4 digits). Add/view/delete.
Notes	Title, content, timestamp	Full notepad with create, edit, and delete. Auto-timestamps every save.
Recovery Codes	2FA code, associated app/website	Store and manage backup codes for two-factor authentication.
Photos	Image files	Import from gallery. Displayed as a thumbnail grid. Tap for full-screen pinch-to-zoom viewer. Long-press for restore/delete options.
Files	Any file type (documents, music, videos, zips)	Import via file picker. Shows file type icon, name, and size. Options to restore to Downloads folder or delete permanently.

File Operations:

- **Restore from Vault:** On native, copies the file back to the device's Downloads folder. On web, triggers a browser download via dart:html Blob/AnchorElement.
- **Delete Permanently:** Removes the hidden file and its metadata entry.

8. Dark Web Scanner (HaveIBeenPwned)

What it does: Checks if the user's passwords have been exposed in known data breaches.

How it works:

- User enters a password into the scan field.
 - The password is **never sent over the network in plain text**. Instead, it uses the **k-Anonymity model**:
 1. The password is hashed locally using **SHA-1** (crypto package).
 2. Only the first 5 characters of the hash are sent to the HIBP API (GET <https://api.pwnedpasswords.com/range/{prefix}>).
 3. HIBP returns all hash suffixes matching that prefix.
 4. The app checks locally if the full hash appears in the returned list.
 - If found, the app displays how many times the password has appeared in breaches.
 - Other scan categories (Email, Phone, Username, Credit Card) show a "Coming Soon" snackbar.
 - Default scan type is set to "Password".
-

9. 🎮 Chonjo (Cybersecurity Quiz Game)

What it does: Gamified cybersecurity education through timed quizzes.

How it works:

- User starts from the **Chonjo Intro Screen** → selects a **difficulty level** → enters the game.
 - Backend loads questions from a curated `chonjo_questions_bank.json` file containing cybersecurity questions.
 - `POST /api/chonjo/daily-quiz?level=N` returns 20 randomized questions for the chosen level.
 - Each question has a **countdown timer**. Answering correctly adds to the score; wrong answers show the correct answer with an explanation.
 - An animated progress bar tracks completion through all 20 questions.
 - On completion, the score is submitted to `POST /api/chonjo/submit-score`, updating the user's `total_score` and `streak` in the database.
 - Streak tracking rewards consistent play.
-

10. 🤖 Cyber Agent (AI Assistant)

What it does: An interactive AI cybersecurity advisor that answers questions, provides guidance, and remembers conversation context.

How it works:

- Chat-style interface with user and agent message bubbles.
 - User sends a message to POST /api/agent/chat.
 - The backend retrieves the user's **last 12 messages** from the chat_messages table to provide conversation context.
 - The message + history are sent to **Google Gemini AI** with a specialized system prompt that positions the agent as a Kenyan cybersecurity expert.
 - Gemini's response is streamed back and displayed as a formatted chat bubble.
 - Both user and agent messages are persisted to the database, enabling continuous conversation across app sessions.
-

11. 🚨 Active Defense (Community Scam Reporting)

What it does: A community-powered database for reporting and checking phone numbers associated with fraud.

How it works:

- **Check a Number:** User enters a phone number. The app queries GET /api/active-defense/check/{phone_number}. If the number exists in the reported_numbers table, the app displays the scam category, report count, and last reported date. If not found, it's marked as safe.
 - **Report a Number:** User enters a phone number, selects a scam category (e.g., "Mobile Money Fraud", "Phishing", "Impersonation"), and optionally adds details. Submitted to POST /api/active-defense/report. If the number already exists, its report_count is incremented.
 - Both features are accessed via popup dialogs styled with the vault theme.
-

12. 🌀 Navigation — Dock Nav Bar

What it does: A custom floating navigation system inspired by macOS dock design.

How it works:

- Sits at the bottom of the screen as a floating bar with animated tile icons.
- **Left side:** Vault, Boma, Mulika.
- **Center:** Home (Dashboard) — highlighted with a unique design.
- **Right side:** Intel, Agent, Chonjo.
- Uses IndexedStack for instant tab switching without rebuilding widget trees.
- Semi-transparent backdrop blur behind the dock creates a glass shelf effect.
- Chonjo is handled separately — it pushes its own Navigator route (Intro → Levels → Game) rather than living in the IndexedStack.

Authentication & Security

Feature	Implementation
User Registration	Email + username + password. Password hashed with bcrypt via Passlib. Stored in PostgreSQL.
Login	Validates credentials, returns a signed JWT token (HS256, 7-day expiry).
Token Storage	JWT stored on-device via flutter_secure_storage (Keystore/Keychain/DPAPI).
Protected Endpoints	All /api/* routes (except auth) require a valid Authorization: Bearer <token> header. Verified via get_current_user dependency injection.
Vault Encryption	All vault metadata (passwords, cards, notes, codes) encrypted at rest via flutter_secure_storage's AES-256 backend.
Password Scanning	Uses SHA-1 k-Anonymity — the full password hash never leaves the device.

Cross-Platform Support

The app is built with a single Flutter codebase and runs on:

Platform	Status	Notes
Android	✅ Full support	All features work natively including real app scanning, biometric auth, and file hiding.
iOS	✅ Supported	Same as Android (via Flutter's iOS compilation).
Web (Chrome/Edge)	✅ Supported	Vault uses base64 in-browser storage. Device scan uses mock data. File restore triggers browser download.
Windows Desktop	✅ Supported	Added flutter_secure_storage_windows and local_auth_windows for native Windows support.
macOS/Linux	🟡 Partial	Should compile but untested.